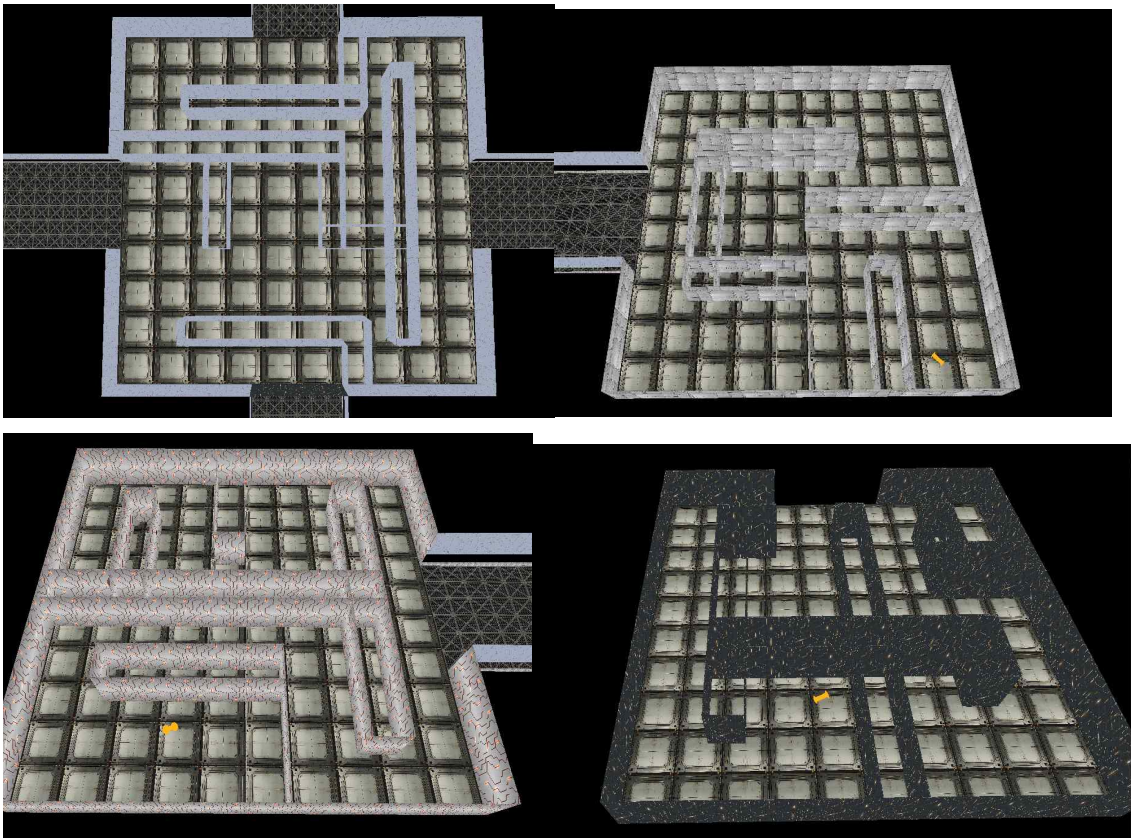


2023년도 컴퓨터 그래픽스 기말 보고서

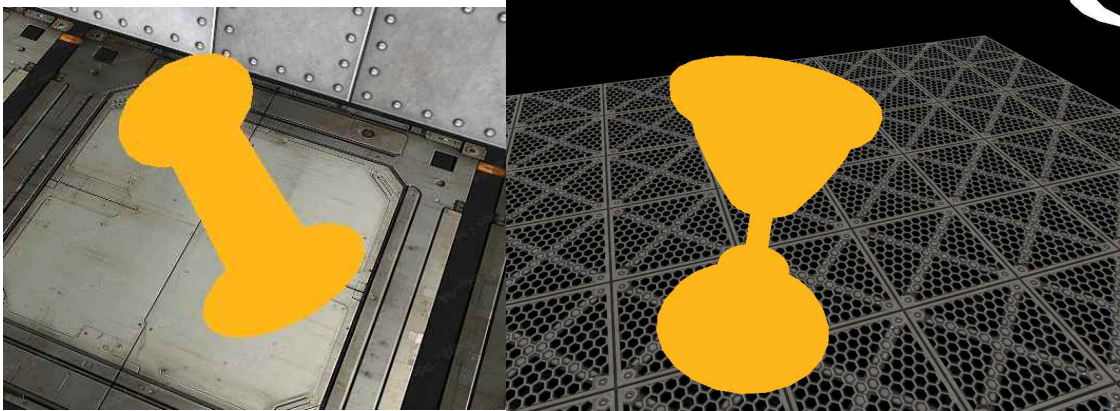
20201772 김재원

1. 미로 구현

미로의 컨셉은 '우주 공간에 있는 정거장'을 모티브로 하여 만들었다. 미로는 크게 중앙, 우측, 좌측, 후방 구역으로 나누어지며 중앙 구역의 정면 쪽은 특정 조건을 만족하기 전까지 장애물로 막혀있는 구조이다. 플레이어의 최종적인 목표는 중앙 구역에서 이어지는 우측, 좌측, 후방 구역에 들어가 각 구역이 숨겨진 세 개의 배터리 오브젝트를 모아 장애물을 해제하고 잠겨진 구역에 있는 트로피를 찾는 것이다.



좌상단 : 중앙 구역, 우상단 : 우측 구역, 좌하단 : 좌측 구역, 우하단 : 후방 구역
중앙 구역은 세 구역의 연결점 역할을 하며 나머지 구역 어딘가에는 목적지로 가기 위한
아이템이 숨겨져 있다



좌측 : 배터리(아이템), 우측 : 트로피

3개의 배터리를 모아 장애물을 해제할 수 있고 그 너머엔 탈출구와 트로피가 있다.

1-1) 카메라(플레이어) 구현

사용자의 키보드 및 마우스 조작을 입력받고 카메라의 위치 및 방향을 바꿀 수 있도록 구현하였다. 사용자는 WASD 키를 눌러 각각 전진, 좌로 이동, 후진, 우로 이동을 할 수 있고 추가적인 편의로써 R키를 눌러 초기 시작 지점으로 즉시 이동할 수 있도록 구현하였다.

추가적인 기능으로는 Q, E키를 활용하여 카메라의 높낮이를 조절하는 기능이 있었으며 실제 게임 상에서는 사용할 수 없는 디버그용 조작으로 활용하였다. 또한 ESC 키를 눌러 게임을 강제로 종료할 수 있다.

1-2) 아이템 및 기타 장식 구현

MapObject라는 클래스를 제작하여 아이템 및 장식에 사용할 변수와 메소드를 일괄적으로 처리할 수 있게 하였다. 사용자는 해당 오브젝트를 생성할 때 위치와 바라볼 방향, 충돌 범위와 사용할 폴리곤 모델 등을 인자로 넣을 수 있다.

```
class MapObject {
public:
    MapObject(const glm::vec3& position, const glm::vec3& direction, float radius, int texture, string polygonName = "")
        : position(position), direction(glm::normalize(direction)), radius(radius), texture(texture), polygonName(polygonName) {
        readModel(polygonName);
    }
    // Getter functions
```

처음 입력받는 두 개의 벡터는 각각 좌표상의 위치와 바라볼 방향을 나타내며 radius 실수는 플레이어와의 충돌에 활용할 범위를 나타낸다. texture 정수는 실제로 사용하지 않으며 마지막으로 사용할 폴리곤 파일의 이름을 문자형 변수로 받아와 저장할 수 있다. 해당 클래스 내부에는 각 변수를 설정하거나 받아올 메소드가 존재하며 이를 통해 변수를 활용 또는 변경할 수 있다.

```
// Rendering function
virtual void render() {
{
    glPushMatrix();
    glm::mat4 model = glm::translate(glm::mat4(1.0f), position);

    // 방향 벡터의 각 성분에 대해 세 번 회전
    model = glm::rotate(model, glm::radians(glm::degrees(atan2(direction.x, direction.z))),
    model = glm::rotate(model, glm::radians(glm::degrees(atan2(direction.y, direction.x))),
    model = glm::rotate(model, glm::radians(glm::degrees(atan2(direction.z, direction.y))),

    glmMultMatrixf(glm::value_ptr(model));
    glColor3f(1.0f, 0.7f, 0.1f);
}
```

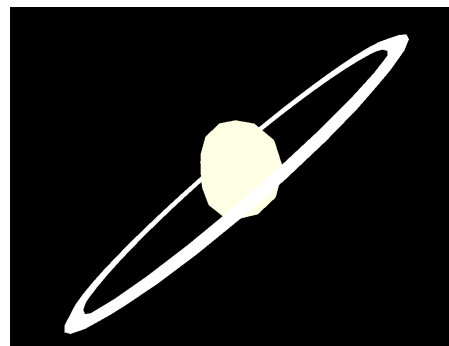
클래스 내부엔 각 오브젝트를 렌더링하는 메소드가 있으며, 이를 display 함수에 넣어 활용하고 있다. 특별히 조정이 필요한 오브젝트의 경우 인자로 받아온 폴리곤의 이름을 검사하여 특정 문장일 경우 조건식 내부의 렌더링을 진행하도록 되어있다.

```
// checkCollision with camera function
virtual bool checkCollisionWithCamera(float cameraX, float cameraY, float cameraZ)
{
    glm::vec3 diff = position - glm::vec3(cameraX, cameraY, cameraZ);
    float dist = glm::length(diff);
    float sumRadius = radius + 2.0f;

    if (dist < sumRadius) {
        return true;
    }
    return false;
}
```

충돌 검사 메소드는 플레이어와의 거리를 계산하고 일정 거리 이하일 경우 참을 반환하도록 설계되어있다. 특정 아이템의 경우 이렇게 받은 boolean 변수를 통해 획득 여부와 아이템의 소멸을 호출하는 구조이며, 배터리 아이템이 여기에 해당한다. 플레이어가 배터리와 부딪히면 아이템이 소멸하며 획득 여부가 참으로 바뀌게 되고 세 개의 배터리가 모두 얻어지면 최종 목적지로 향하는 장애물을 해제한다.

그 외에도 장식에 쓰이는 미로 밖의 행성들의 경우도 해당 클래스를 활용하여 폴리곤 모델을 불러와 렌더링한 것이다.



1-3) 벽, 바닥, 천장 구현

PlaneObject는 앞서 서술한 MapObject를 상속받은 객체이며 부모 객체의 변수 및 메소드를 활용하면서도 추가적인 기능을 할 수 있도록 설계되었다.

```
class PlaneObject : public MapObject {
public:
    PlaneObject(const glm::vec3& position, const glm::vec3& direction, int texture, const glm::vec3& direction_height, float width, float height)
        : MapObject(position, direction, 0.0f, texture), direction_height(glm::normalize(direction_height)), width(width), height(height) {
        // 만약 direction과 direction_height가 서로 수직이 아니라면 경고 메시지를 출력하고 direction_height를 direction에 수직인 벡터로 만든다.
        if (glm::dot(direction, direction_height) != 0.0f) {
            std::cout << "Warning: direction and direction_height are not perpendicular!" << std::endl;
            this->direction_height = glm::normalize(glm::cross(direction, direction_height));
        }
        initTexture();
    }
};
```

부모 클래스보다 하나 더 많은 벡터를 인자로 받을 수 있는데, 이는 사각형을 그릴 때 필요한 방향을 정하기 위해 설정한 것이다. 세 개의 벡터는 각각 공간상에 위치할 좌표와 사각형을 그릴 때 필요한 너비와 높이의 방향을 지정하며 사용할 텍스처는 정수로 받아 그 값이 얼마인지에 따라 적용할 이미지와 반복 패턴을 결정한다. 이후 평면의 너비와 높이를 실수 변수로 받아 그려질 사각형의 크기를 정할 수 있게 하였다.

```
void initTexture() {
    if (textureID != 0) {
        glDeleteTextures(1, &textureID);
    }
    const char* textureName;

    // 인자로 받은 정수에 따라 텍스처를 다르게 설정한다.
    switch (getTexture())
    {
    case 1:
        textureName = "../bin/Texture/dia_panel.bmp";
        break;

    case 2:
        textureName = "../bin/Texture/metal_panel.bmp";
        break;

    case 3:
        textureName = "../bin/Texture/Pcircuit.bmp";
        break;
    }
```

미로 내의 여러 구역이 시각적으로 쉽게 구분할 수 있게 만들고자 하였고 맵을 구성하는 여러 평면이 고유의 텍스처를 가지게 만드는 것이 좋겠다고 생각하였다. 이에 입력받은 Texture 정수가 무엇인지 따라 적용할 이미지의 파일을 결정하도록 했으며 같은 텍스처라도 다른 정수값으로 설정하여 렌더링 메소드에서 다른 방식을 적용하게끔 만들어 상이한 반복 패턴을 생성하게 하였다.


```
// 충돌에 따라 카메라의 이동 방향 및 거리를 결정하는 함수
glm::vec3 collisionNavigation(float cameraX, float cameraY, float cameraZ, glm::vec3 direction) {

    float deltaX = cameraX + direction.x;
    float deltaY = cameraY;
    float deltaZ = cameraZ + direction.z;
    glm::vec3 moveDirection = direction;

    glm::vec3 normal = glm::normalize(glm::cross(getDirection(), getDirection_height()));
    float dist_position = glm::abs(glm::dot(getPosition() - glm::vec3(deltaX, deltaY, deltaZ), normal));
    float dist_direction = glm::abs(glm::dot(getDirection(), glm::vec3(deltaX, deltaY, deltaZ) - getPosition()));
    float dist_direction_height = glm::abs(glm::dot(getDirection_height(), glm::vec3(deltaX, deltaY, deltaZ) - getPosition()));

    if (dist_position < 2.0f && dist_direction < width + 2 && dist_direction_height < height + 2) {
        moveDirection = glm::dot(moveDirection, getDirection()) * getDirection();
        return moveDirection;
    }
    return moveDirection;
}
```

PlaneObject 내부에는 카메라와의 충돌을 검사하고 충돌이 일어났을 경우 카메라의 이동 방향과 그 거리를 변경할 수 있게 만들었다. 이 메소드 카메라의 위치와 방향을 인자로 받고 해당 클래스의 위치와 크기를 기반으로 생성된 벡터와의 외적으로 충돌이 일어났는지의 여부를 검사한다. 충돌이 감지되면 입력받은 카메라의 방향을 벽과 평행하게 만들고 그 속도를 벽 벡터와의 내적 값만큼 곱한다. 이를 통해 플레이어는 벽을 통과하지 못하며 대신 벽과 평행하게 밀려나게 된다.

```
void keyboard(unsigned char key, int x, int y) {
    // 카메라 이동 속도 및 방향 변수
    const float cameraSpeed = 0.9f;
    float moveX = cameraSpeed * sin(cameraYaw);
    float moveZ = -1 * cameraSpeed * cos(cameraYaw);

    // 구역 생성 체크
    static bool isMainGenerated = false;
    static bool isRightGenerated = false;
    static bool isLeftGenerated = false;
    static bool isBackGenerated = false;
    static bool isFrontGenerated = false;

    switch (key) {
        case 'w': // 전진
        {
            glm::vec3 moveDirection = collisionCalculation(cameraX, cameraY, cameraZ, moveX, moveZ);
            cameraX += moveDirection.x;
            cameraZ += moveDirection.z;
            break;
        }
    }
}
```

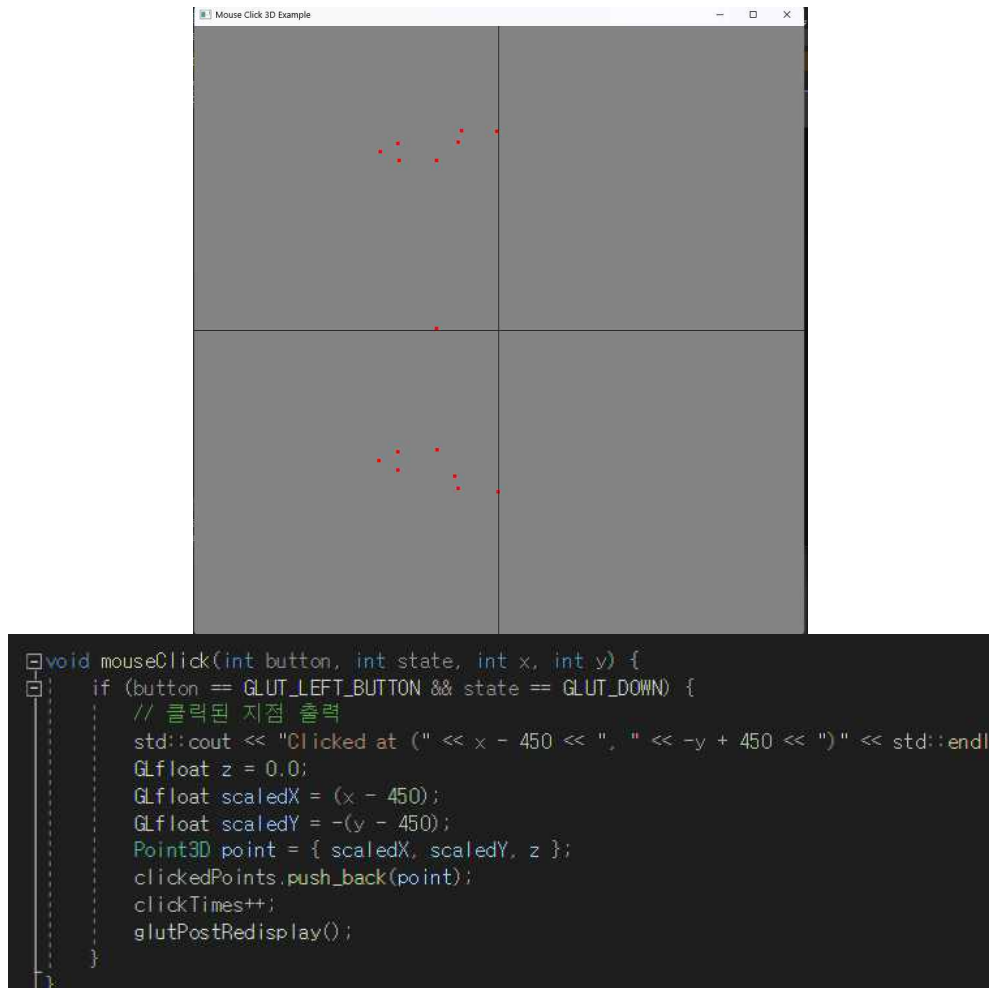
해당 메소드는 키보드 입력 함수에서 불러와지며, 일정 범위 내의 모든 벽에 대하여 충돌을 검사하고 카메라의 움직임을 계산할 수 있게 만들어졌다. 맵 내의 모든 벽에 대해 검사하는 것은 성능 저하가 심하여 일정 범위 내의 벽에 대해서만 그 기능을 수행하게 하였다.

2. SOR 모델러 구현

사용자가 화면에 클릭을 하면 그 지점에 점을 찍고 마우스 우클릭을 통해 찍은 점들을 회전시키고 그 인덱스를 일정한 규칙에 따라 구조체의 배열로 생성할 수 있게 하였다. 그렇게 나

열된 구조체는 폴리곤을 제작할 순서로 활용되며 저장 기능을 통해 각 정점과 정점의 연결 순서를 dat로 저장하는 프로그램이다.

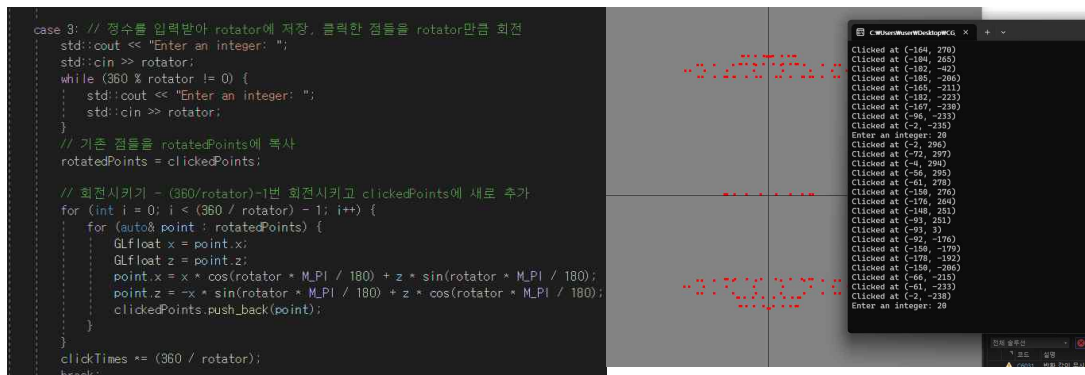
2-1) 점 생성 구현



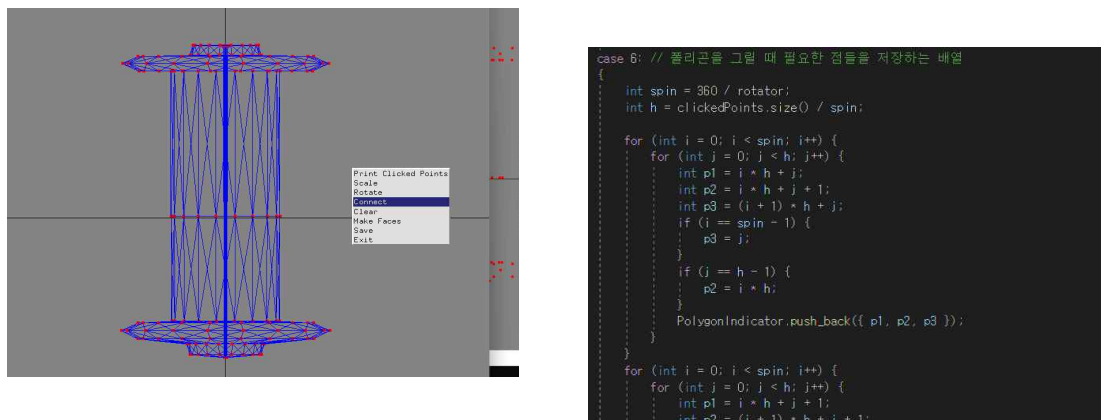
사용자가 화면 상에 클릭을 하며 해당 좌표에 빨간색 점을 출력하게 하였다. 클릭한 좌표는 3차원 공간상의 좌표를 나타내는 구조체에 들어가며 클릭을 반복할 때마다 해당 기능을 반복하여 수행한다. 사용자는 프로그램 화면에서 우클릭을 통해 메뉴를 띄울 수 있으며 메뉴마다 가진 고유의 기능을 통해 클릭한 점을 활용할 수 있다.

2-2) 정점 회전

점을 찍은 이후 사용자가 메뉴에서 rotate를 선택하면 cmd 창을 통해 정수 값을 입력할 수 있다. 해당 값이 360의 약수가 아닐 경우 그 값을 다시 입력받으며 360의 약수면 그 각도만큼 클릭된 좌표를 y축을 기준으로 회전시킨다. 회전된 점들은 정점을 저장하는 구조체에 들어가며 이 과정이 360 한 바퀴를 돌 때까지 반복된다.



2-3) 정점의 순서도(Face) 생성 및 정점 연결



사용자는 Make Faces란 메뉴를 선택하여 각 정점을 연결하는 순서도인 Face를 생성할 수 있다. 해당 프로그램의 실행 동안 Face라는 정수 세 개를 저장하는 구조체가 존재하며 위 메뉴를 선택하면 찍은 점과 회전한 각도에 따른 반복으로 찍어놓았던 정점의 인덱스를 구조체에 삽입한다. 이후 connect 메뉴를 선택하면 Face에 들어있는 인덱스 값에 맞추어 정점을 연결하고 이를 화면에 보여주어 최종적으로 완성될 폴리곤의 모습을 시연한다.

2-4) 저장 및 기타 기능

```

case 7: // PolygonIndicator에 저장된 구조체를 dat 파일에 저장
{
    FILE* fp = fopen("polygon.dat", "w");
    fprintf(fp, "VERTEX = %d\n", clickedPoints.size());
    for (const auto& point : clickedPoints) {
        fprintf(fp, "%f %f %f\n", point.x, point.y, point.z);
    }
    fprintf(fp, "FACE = %d\n", PolygonIndicator.size());
    for (const auto& polygon : PolygonIndicator) {
        fprintf(fp, "%d %d %d\n", polygon.p1, polygon.p2, polygon.p3);
    }
    fclose(fp);
    break; }

case 8:
    // 프로그램 종료
    exit(0);
}

```

Face 구조체를 전부 생성하였다면 메뉴에서 save를 눌러 찍어 놓은 정점과 그 순서도인 Face를 .dat 형식의 파일로 저장할 수 있다. 그 외의 부가 기능의 경우, 저장된 정점의 좌표를 확인할 수 있는 기능과 각 정점의 위치를 실수 값만큼 확대 및 축소할 수 있는 scale, 모든 객체를 초기화하고 처음부터 프로그램을 실행할 수 있는 clear와 프로그램을 종료하는 exit 등이 존재한다.

3) 어려웠던 점 및 느낀 점

첫 번째로 어려웠던 것은 벽과 카메라의 충돌 구현이었다. 이론적으로 외적 활용만 하면 쉬울 것 같았으나, 해당 기능을 벽에 넣을지 카메라에 넣을지 그것도 아니면 외부적으로 처리해야 할지 그 구조를 구상하는 것이 꽤 어려웠다. 단순히 하나의 벽만 존재하는 것이 아니고 플레이어의 위치는 계속 변하므로 충돌의 검사와 그에 따른 이동 제한을 구현하는 것이 많이 힘들었던 것 같다.

두 번째로 어려웠던 것은 최적화이다. 기능 구현을 우선하여 프로그램을 만들고 있었는데 어느 순간 알 수 없는 이유로 텍스처가 무작위로 변경되는 이슈가 발생하였고 여러 검사를 하다 메모리 문제라고 결론이 지어져 객체 생성과 각종 기능 시행에 최적화를 시행하여야 했다.

```

if (cameraX < 100.0f && cameraX > -100.0f && cameraZ < 100.0f && cameraZ > -100.0f) {
    for (int i = 0; i < wallsMain.size(); i++) {
        moveDirection = wallsMain[i]->collisionNavigation(cameraX, cameraY, cameraZ, moveDirection);
    }
    for (int i = 0; i < wallsLock.size(); i++) {
        moveDirection = wallsLock[i]->collisionNavigation(cameraX, cameraY, cameraZ, moveDirection);
    }
}

if (cameraZ > 40.0f) {
    for (int i = 0; i < wallsRight.size(); i++) {
        moveDirection = wallsRight[i]->collisionNavigation(cameraX, cameraY, cameraZ, moveDirection);
    }
}

if (cameraZ < -40.0f) {
    for (int i = 0; i < wallsLeft.size(); i++) {
        moveDirection = wallsLeft[i]->collisionNavigation(cameraX, cameraY, cameraZ, moveDirection);
    }
}

```


위의 코드가 그 예시인데 렌더링 및 충돌 검사를 모든 벽 객체에 진행하는 것이 불가능하다고 생각하여 카메라 위치에 따라 객체 생성 및 함수 시행의 제약을 넣게 되었다. 가벼운 프로그램을 만든다고 생각하였으나 강제적으로 최적화를 진행하게 될 줄은 생각도 못하였다.

해당 프로젝트를 진행하며 깨달은 점은 프로젝트가 거대해짐에 따라 프로그래밍에 신경 써야 하는 것이 굉장히 많다는 것이다. 단순 논리 구조뿐만 아니라 프로젝트의 종속성과 파일 경로, 라이브러리 연결과 클래스의 상속과 상/하위 객체 분류 등 많은 것을 고민해야 함을 알게 되었다. 시작할 때 무작정 헤더파일을 많이 만들었다가 코드가 온통 꼬였던 것을 생각하면 큰 프로젝트를 진행할 때 그 시작부터 신중해야 했던 것 같다. 더 하고 싶은 것도 많고 아쉬움도 꽤 있지만 많은 것을 배운 프로젝트였다.